INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Tecnología de Objetos				
TÍTULO DE LA PRÁCTICA:	Puntero en C++				
NÚMERO DE PRÁCTICA:	02	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VI (sexto)
FECHA DE PRESENTACIÓN	11/09/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): Franco Jesus Cahua Soto				**NOTA:**	
DOCENTE(s): Mg. Ing. Rene Alonso Nieto Valencia					
Modalidad: Individual		Inicio de la práctica: 07/09/2026			

SOLUCIÓN Y RESULTADOS

I. INTRODUCCIÓN

El dominio de los punteros y la administración directa de memoria constituyen dos de los pilares más determinantes del lenguaje C++ y de la ingeniería de software de alto rendimiento. A diferencia de lenguajes con recolección automática de basura (*Garbage Collection*), C++ confiere al desarrollador la potestad y la responsabilidad de gestionar el ciclo de vida de los recursos computacionales mediante asignación estática y dinámica.

En la presente práctica de laboratorio **N.° 02** de la asignatura **Tecnología de Objetos**, se estudia de forma exhaustiva la semántica de punteros (*pointers*), la dualidad arquitectónica entre memoria de pila (**Stack**) y memoria de montón (**Heap**), los operadores de indirección y referencia, y el diseño e implementación de estructuras de datos autorreferenciadas, tales como listas doblemente enlazadas y sistemas modulares orientados a objetos.

SOLUCIÓN Y RESULTADOS

I.1 Objetivos de la práctica

- Comprender los mecanismos de direccionamiento de memoria, desreferenciación y aritmética de punteros en C++.
- Diferenciar el ciclo de vida de variables asignadas en Stack frente a memoria dinámica asignada en Heap mediante los operadores `new` y `delete`.
- Implementar una arquitectura modular de tres clases para una calculadora matemática con análisis léxico, motor aritmético y despachador.
- Desarrollar una estructura de datos lineal autorreferenciada: Lista Doblemente Enlazada con gestión dinámica de memoria.
- Diseñar formalmente un algoritmo en pseudocódigo para la evaluación de polinomios mediante listas enlazadas.

II. MARCO CONCEPTUAL

II.1 Punteros y Direccionamiento de Memoria

La memoria física de un computador se estructura como un arreglo unidimensional contiguo de celdas de un byte, cada una asociada a una dirección hexadecimal única. Un **puntero** es una variable cuyo valor es precisamente la dirección de memoria de otra variable o bloque de datos:

$$p = \&x \implies {}^{*}p \equiv x \tag{1}$$

donde:

- $\&$ es el **operador de dirección**: retorna la posición en memoria asignada a su operando.
- $*$ es el **operador de indirección o desreferenciación**: permite acceder y modificar el contenido alojado en la dirección a la que apunta el puntero.

II.2 Arquitectura de Memoria: Stack vs. Heap

El espacio de direccionamiento virtual de un proceso en ejecución se organiza fundamentalmente en dos segmentos de memoria para variables:

SOLUCIÓN Y RESULTADOS

Criterio	Memoria Stack (Pila)	Memoria Heap (Montón)
Gestión de ciclo	Automática (por ámbito de función / bloque).	Manual por el programador (`new` y `delete`).
Velocidad	Extremadamente veloz (movimiento del puntero RSP).	Moderada (búsqueda de bloques y fragmentación).
Crecimiento	Crece hacia direcciones decrecientes de memoria.	Crece hacia direcciones crecientes de memoria.
Tamaño	Fijo y limitado (típicamente 1 MB a 8 MB).	Limitado únicamente por la memoria física / swap.
Riesgos	Desbordamiento de pila (*Stack Overflow*).	Fugas de memoria (*Memory Leaks*), punteros colgantes.

II.3 Ciclo de Vida Dinámico: Operadores `new` y `delete`

El operador `new` reserva memoria en el Heap para el tipo especificado, invoca el constructor correspondiente y retorna un puntero tipado hacia dicha dirección. Cuando el recurso deja de ser necesario, debe invocarse obligatoriamente `delete` sobre el puntero para invocar el destructor y retornar la memoria al gestor del sistema operativo:

Listado 1: Reserva y liberación dinámica en Heap

```
int* ptr = new int(42);    // Reserva e inicializacion en Heap
std::cout << *ptr;         // Acceso al valor referenciado
delete ptr;                // Liberacion obligatoria
ptr = nullptr;             // Neutralizacion de puntero colgante (dangling pointer)
```

II.4 Acceso a Clases mediante Punteros

Dado un puntero `ptr` a una instancia de una clase u objeto, el acceso a sus miembros puede expresarse mediante dos sintaxis equivalentes:

$$(*ptr).\text{metodo}() \iff ptr \rightarrow \text{metodo}() \tag{2}$$

El operador flecha (->) simplifica la legibilidad al unificar la desreferenciación previa y la selección del miembro.

III. ANÁLISIS DE EJERCICIOS RESUELTOS POR EL DOCENTE

Durante la sesión se evaluaron fragmentos de código fundamentales para observar la interacción con la memoria:

SOLUCIÓN Y RESULTADOS

III.1 Inspección de direcciones y tamaños

Se comprobó que variables con tipos de dato distintos (`shortVar`, `longVar`, `sVar`) ocupan regiones contiguas en el Stack, y que el operador `&` expone sus direcciones hexadecimales relativas.

III.2 Reasignación y mutabilidad de punteros

Se verificó el comportamiento de un puntero `USHORT* pAge` apuntando sucesivamente a `myAge` y posteriormente a `yourAge`. Al alterar `*pAge = 7;`, se constató que la variable original sufre la mutación directa en memoria.

III.3 Gestión dinámica de memoria con `new` y validación

Se analizó la asignación dinámica en Heap:

Listado 2: Fragmento docente de asignacion en Heap

```
int* pHeap = new int;
if (pHeap == NULL) {
    cout << "Error! No memory for pHeap!!";
    return 0;
}
*pHeap = 7;
cout << "*pHeap: " << *pHeap << endl;
delete pHeap;
```

En C++ moderno, si `new` falla en asignar memoria, lanza la excepción estándar `std::bad_alloc`, a menos que se invoque con `std::nothrow`.

III.4 Punteros a objetos y métodos

Se examinó la clase `C` con miembro entero `int x` y puntero `int* p = &x;`. Al instanciar `C c1; C* cptr = &c1;`, se confirmó que tanto `c1.fun()` como `(*cptr).fun()` y `cptr->fun()` invocan el mismo registro de activación del método.

IV. DESARROLLO DE EJERCICIOS PROPUESTOS

IV.1 Ejercicio 1: Calculadora Modular con 3 Clases

Enunciado: Implementar una calculadora con 3 clases en C++, donde:

1. La primera analizará la operación matemática (suma, resta...).
2. La segunda administrará las operaciones matemáticas (núcleo de la calculadora).
3. La tercera procesará la operación ingresada.

El programa recibirá de entrada una cadena de texto (ej. `"10+37"`, `"45+14-42"`, `"1+2+3+4+5+6"`). Como

SOLUCIÓN Y RESULTADOS

máximo el programa recibe 6 números a operar.

Arquitectura Orientada a Objetos:

- `AnalizadorOperacion` **(Parser / Tokenizador):** Escanea la cadena de texto, valida la sintaxis, detecta operadores no soportados o consecutivos, y extrae los operandos en un `std::vector<double>` y operadores en un `std::vector<char>`. Verifica rigurosamente la regla de negocio de admitir como máximo 6 números.
- `AdministradorOperaciones` **(Engine Aritmético):** Encapsula las operaciones primitivas (`sumar`, `restar`, `multiplicar`, `dividir`) y gestiona el control de excepciones ante división por cero.
- `ProcesadorCalculadora` **(Coordinador y Despachador):** Integra los componentes anteriores, resuelve las operaciones respetando la precedencia de operadores y muestra la traza de ejecución paso a paso.

Listado 3: Orquestacion del calculo en ejercicio1.cpp

```
class ProcesadorCalculadora {
private:
    AnalizadorOperacion analizador;
    AdministradorOperaciones administrador;
public:
    bool procesar(const std::string& exp, double& res) {
        if (!analizador.analizar(exp)) return false;
        auto ops = analizador.getOperandos();
        auto opers = analizador.getOperadores();

        // Fase 1: Precedencia de multiplicacion y division
        for (size_t i = 0; i < opers.size(); ) {
            if (opers[i] == '*' || opers[i] == '/') {
                ops[i] = administrador.ejecutarOperacion(opers[i], ops[i], ops[i+1]);
                ops.erase(ops.begin() + i + 1);
                opers.erase(opers.begin() + i);
            } else ++i;
        }
        // Fase 2: Sumas y restas de izquierda a derecha
        while (!opers.empty()) {
            ops[0] = administrador.ejecutarOperacion(opers[0], ops[0], ops[1]);
            ops.erase(ops.begin() + 1);
            opers.erase(opers.begin());
        }
        res = ops[0];
        return true;
    }
};
```

Validación Experimental con las entradas de la guía:

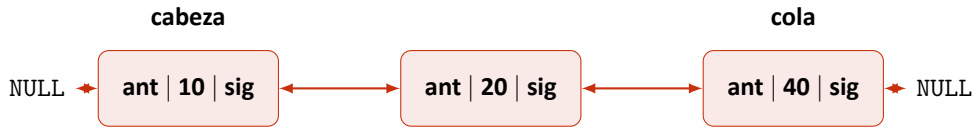
SOLUCIÓN Y RESULTADOS

Cadena de Entrada	N.° Números	Resultado	Estado / Diagnóstico
"10+37"	2	47.0000	Éxito (Suma simple)
"45+14-42"	3	17.0000	Éxito (Suma y resta encadenada)
"1+2+3+4+5+6"	6	21.0000	Éxito (Límite máximo permitido)
"100-25+15-10+50"	5	130.0000	Éxito (Evaluación secuencial)
"10*5+20/4-2"	5	53.0000	Éxito (Precedencia jerárquica)
"1+2+3+4+5+6+7"	7	—	Rechazado (Excede el límite de 6 números)

IV.2 Ejercicio 2: Lista Doblemente Enlazada con Punteros

Enunciado: Implementar con punteros una lista doblemente enlazada, utilizar clases o struct.

Diseño de la estructura de datos: Cada nodo de la lista almacena un valor entero (`int dato`) y dos punteros: `ant` (apunta al nodo predecesor) y `sig` (apunta al nodo sucesor). La clase `ListaDoblementeEnlazada` gestiona punteros a la `cabeza` (*head*) y a la `cola` (*tail*), garantizando operaciones eficientes en ambos extremos con complejidad $O(1)$.



Listado 4: Inserción ordenada y eliminación en ejercicio2.cpp

```
void insertarEnOrden(int valor) {
    if (estaVacia() || valor <= cabeza->dato) { insertarInicio(valor); return; }
    if (valor >= cola->dato) { insertarFinal(valor); return; }
    NodoDoble* actual = cabeza;
    while (actual != nullptr && actual->dato < valor) actual = actual->sig;
    NodoDoble* nuevo = new NodoDoble(valor);
    nuevo->sig = actual;
    nuevo->ant = actual->ant;
    actual->ant->sig = nuevo;
    actual->ant = nuevo;
    ++cantidad;
}

bool eliminar(int valor) {
    NodoDoble* act = cabeza;
    while (act != nullptr && act->dato != valor) act = act->sig;
    if (act == nullptr) return false;
    if (act == cabeza && act == cola) cabeza = cola = nullptr;
    else if (act == cabeza) { cabeza = cabeza->sig; cabeza->ant = nullptr; }
    else if (act == cola) { cola = cola->ant; cola->sig = nullptr; }
    else { act->ant->sig = act->sig; act->sig->ant = act->ant; }
    delete act;
    --cantidad;
    return true;
}
```

SOLUCIÓN Y RESULTADOS

Pruebas de recorrido bidireccional y eliminación:

- **Recorrido hacia adelante (Cabeza → Cola):**
  `NULL <->[10] <->[20] <->[40] <->NULL`
- **Recorrido inverso hacia atrás (Cola → Cabeza):**
  `NULL <->[40] <->[20] <->[10] <->NULL`
- **Gestión de memoria:** El destructor `ListaDoblementeEnlazada()` itera secuencialmente invocando `delete` en cada nodo, garantizando la eliminación absoluta de fugas de memoria.

V. CUESTIONARIO

V.1 Pseudocódigo para resolución de polinomios usando listas enlazadas

Enunciado: Proponer un pseudocódigo sobre la resolución de polinomios usando una lista enlazada (ej. $f(X) = 7X^2 + 2X + 1$). El pseudocódigo debe guardar los resultados para diferentes valores de X.

V.1.1 Estructura de Datos Propuesta

- `NodoTermino`: Almacena cada monomio del polinomio con campos `coeficiente` (real), `exponente` (entero) y puntero `siguiente`.
- `NodoResultado`: Almacena el par ordenado $(\text{valorX}, \text{evaluacionFX})$ y el puntero `siguiente`.

V.1.2 Algoritmo en Pseudocódigo Formal

Listado 5: Pseudocódigo de evaluación y almacenamiento de polinomios

```
ALGORITMO EvaluacionPolinomioEnlazado
ESTRUCTURA NodoTermino:
    coeficiente: REAL
    exponente: ENTERO
    siguiente: PUNTERO A NodoTermino
FIN_ESTRUCTURA

ESTRUCTURA NodoResultado:
    x: REAL
    fx: REAL
    siguiente: PUNTERO A NodoResultado
FIN_ESTRUCTURA

// Funcion para evaluar f(x) recorriendo la lista enlazada de terminos
FUNCION EvaluarPolinomio(cabezaPoli: PUNTERO A NodoTermino, valorX: REAL) -> REAL
    acumuladorFX <- 0.0
    actual <- cabezaPoli
    MIENTRAS actual <> NULO HACER
        acumuladorFX <- acumuladorFX + (actual.coeficiente * (valorX ^ actual.exponente))
        actual <- actual.siguiente
    FIN_MIENTRAS
```

SOLUCIÓN Y RESULTADOS

```
    RETORNAR acumuladorFX
FIN_FUNCION

// Procedimiento para procesar un conjunto de valores de X y guardar resultados
PROCEDIMIENTO ProcesarValoresX(cabezaPoli: PUNTERO A NodoTermino,
                               valoresX: ARREGLO DE REAL, n: ENTERO,
                               cabezaRes: REF PUNTERO A NodoResultado)
    PARA i <- 1 HASTA n HACER
        valX <- valoresX[i]
        resFX <- EvaluarPolinomio(cabezaPoli, valX)

        // Crear nuevo nodo de resultado dinamicamente
        nuevoNodoRes <- NUEVO NodoResultado
        nuevoNodoRes.x <- valX
        nuevoNodoRes.fx <- resFX
        nuevoNodoRes.siguiente <- NULO

        // Insertar al final de la lista de resultados
        SI cabezaRes = NULO ENTONCES
            cabezaRes <- nuevoNodoRes
        SINO
            temp <- cabezaRes
            MIENTRAS temp.siguiente <> NULO HACER
                temp <- temp.siguiente
            FIN_MIENTRAS
            temp.siguiente <- nuevoNodoRes
        FIN_SI
    FIN_PARA
FIN_PROCEDIMIENTO
```

V.1.3 Análisis de Complejidad

- **Complejidad Temporal:** Evaluar un polinomio con T términos para M valores de X consume $O(T)$ operaciones por valor, totalizando un costo asintótico óptimo de $O(M \cdot T)$.
- **Complejidad Espacial:** Almacena T nodos en el Heap para los términos y M nodos para los resultados, con un consumo de memoria auxiliar de $O(T + M)$.

VI. CONCLUSIONES

1. Los punteros en C++ permiten el control directo de las direcciones físicas de memoria, otorgando máxima eficiencia computacional al evitar copias innecesarias de estructuras complejas.
2. La separación entre Stack y Heap es fundamental para el diseño arquitectónico: el Stack provee asignación ultrarrápida y gestión automática por ámbito, mientras que el Heap posibilita la persistencia dinámica de datos no acotados en tiempo de compilación.
3. La división de responsabilidades en 3 clases (`AnalizadorOperacion`, `AdministradorOperaciones` y `ProcesadorCalculadora`) ilustra el principio de responsabilidad única (SRP), desacoplando la validación

SOLUCIÓN Y RESULTADOS

de entrada de la computación aritmética.

4. Las listas doblemente enlazadas maximizan la versatilidad de manipulación de datos lineales al permitir recorridos bidireccionales e inserciones/eliminaciones en tiempo constante $O(1)$ una vez localizado el nodo, demandando siempre la liberación sistemática de cada nodo con `delete`.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Universidad Nacional de San Agustín. (2026). *Guía de Práctica de Laboratorio 02: Puntero en C++*. Escuela Profesional de Ingeniería de Sistemas. Código: GUIA-PRLD-001. Docente: Mg. Ing. Rene Alonso Nieto Valencia.

[2] Stroustrup, B. (2013). *The C++ Programming Language* (4.ª ed.). Addison-Wesley Professional.

[3] Deitel, P., y Deitel, H. (2017). *C++ How to Program* (10.ª ed.). Pearson.

[4] Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley Professional.

[5] Meyers, S. (2005). *Effective C++: 55 Specific Ways to Improve Your Programs and Designs* (3.ª ed.). Addison-Wesley.